Knowledge Based Engineering (KBE)
AE4204

Dr.ir. G. La Rocca

Flight Performance & Propulsion

Group (FPP)

Lecture 4

The KBE OO programming approach

- The ParaPy class definition

- exe: Code → UML / UML → code

Contents

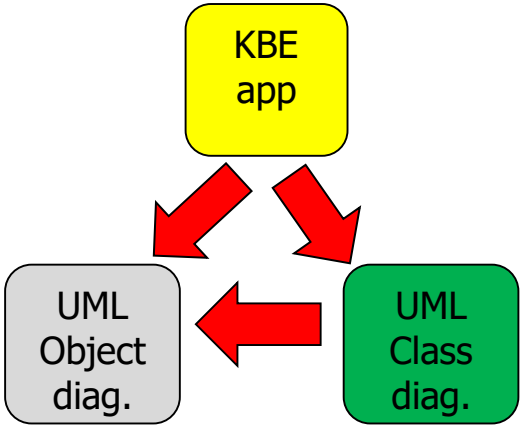
```
class Aircraft(Base):

    # required input slot
    attr1 = Input()
    # optional input slot
    attr2 = Input(value)
    ...
    @Attribute
    def attr3(self):
        return ...

    @Part
    def my_wing(self):
        return Wing(span=10)

    @Part
    def my_fuselage(self):
        return Fuselage(diameter=2, length=8)
```

- The KBE programming approach and KBE languages
- KBE constructs for product model definition
  - The ParaPy `class … (Base)`
- UML modeling exercises:
  - From code to class diagrams
  - From code to object diagrams
  - From object diagram to class diagram



The KBE programming approach:

KBE systems and KBE languages

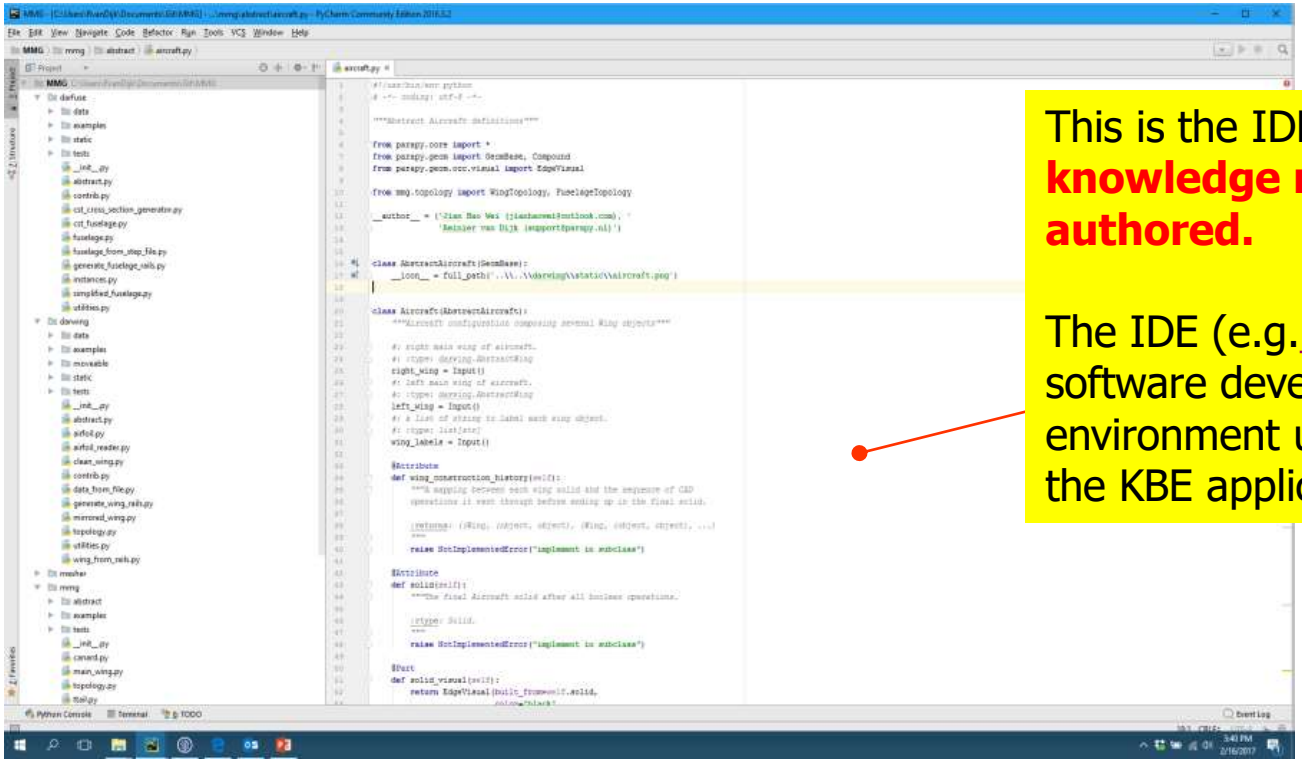
The KBE programming approach

KBE systems provide developers with an **object oriented (OO) programming environment** to model product and process knowledge, including **geometry**, as scalable assemblies of classes and objects.

KBE systems:

- provide an **IDE for developing code-based knowledge models,** rather than a rule-entry and frame form user interface like in FBSs*
- do **not primarily rely on interactive geometry modelling** like CAD tools, but enable geometry creation and manipulation **programmatically** through tightly integrated CAD kernel libraries
- translate the written code via **a compiler/interpreter** into executable engineering workflows**
- provide **runtime user interfaces to interact with the generated product model,** not to pose queries and run the inference engine

The KBE programming approach



This is the IDE, **where the knowledge model is authored.**

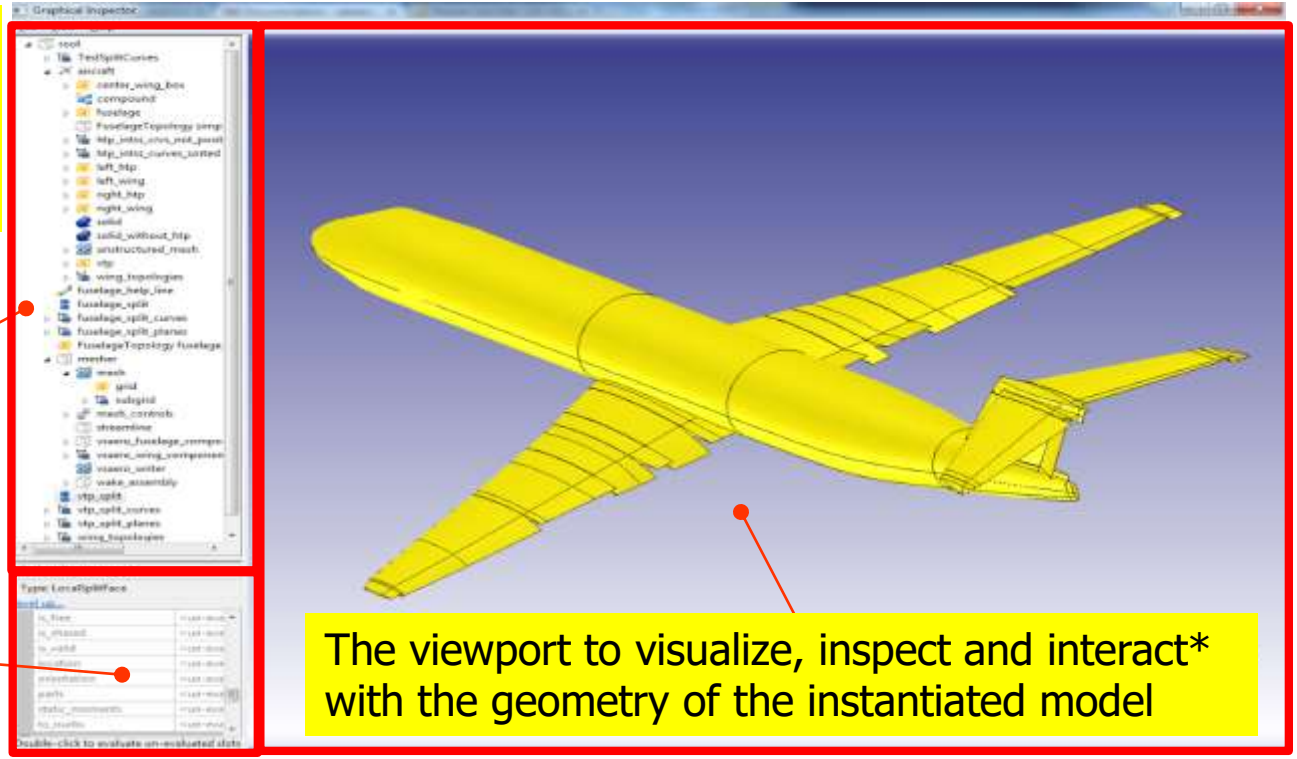
The IDE (e.g. PyCharm) is the software development environment used to program the KBE application

The KBE programming approach

The KBE runtime UI, where the **knowledge model is executed and explored**

The pane where the product tree of the instantiated model is displayed

The property grid, where attribute values can be inspected and input values interactively modified



The viewport to visualize, inspect and interact* with the geometry of the instantiated model

The KBE programming approach

Most KBE systems offer(ed)
LISP as a programming language,
or some other "*lispy*" type of language.

Why Lisp ?

- AI heritage
- strong symbolic capabilities
- OO extensions
- mature in the 1980s–90s
- …

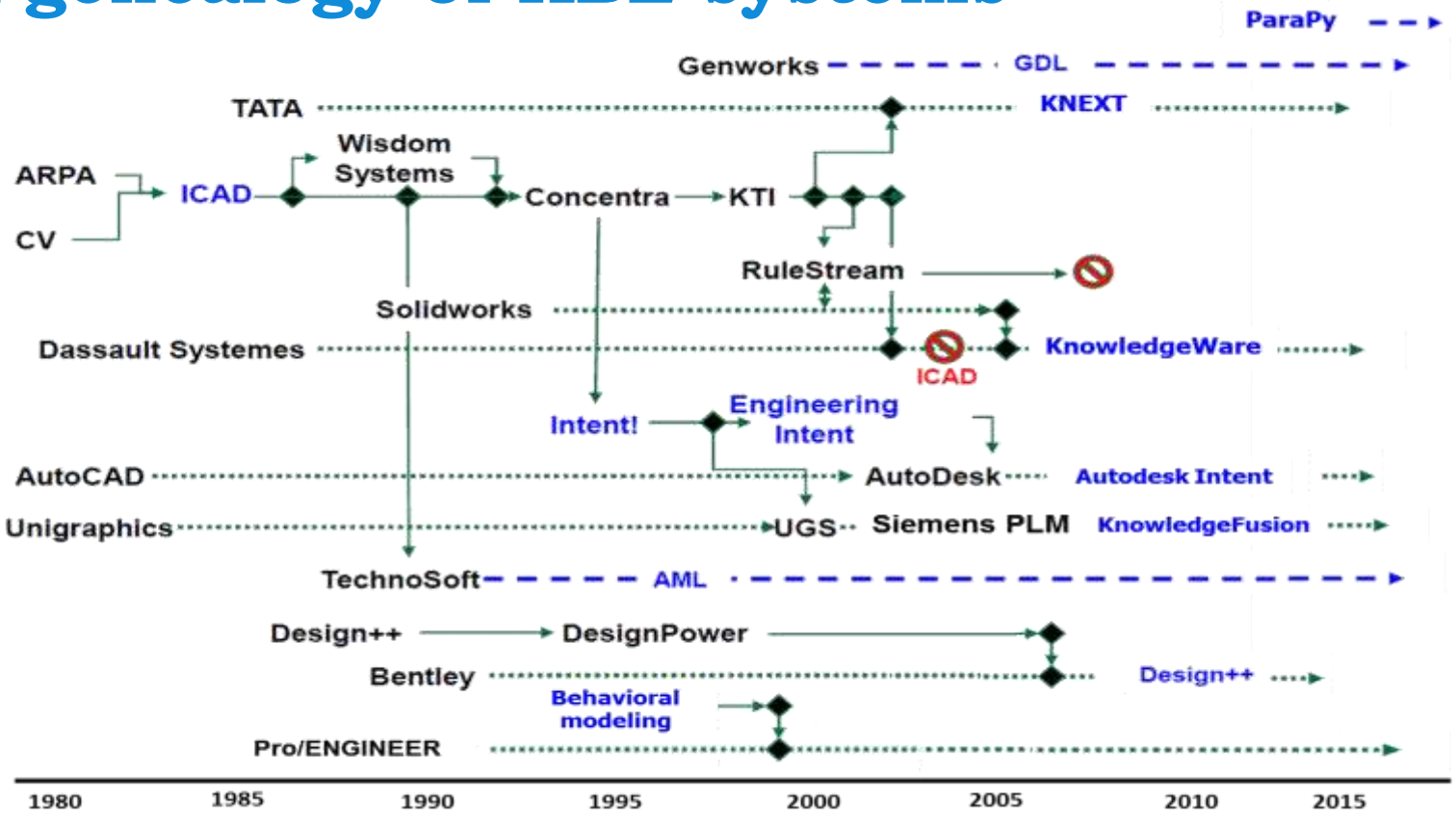
ParaPy

python

Some existing (or existed) KBE languages

1. IDL, the **ICAD** Design Language. It was based on Common LISP, a dialect of LISP, including the CLOS (Common LISP Object System) object-oriented facility.
2. **GDL**, the General-purpose Declarative Language by Genworks' KBE system, is based on the ANSI standard version of Common LISP.
3. **AML**, the Adaptive Modeling Language of Technosoft, was originally written in Common LISP, though, subsequently recoded in a proprietary–yet–LISP-similar language.
4. **Intent!** the KBE proprietary language developed by Heide Corporation and now integrated in the Siemens' NX Knowledge Fusion, belongs also to the family of LISP-inspired languages.
5. **Para*Py***, a Python-based KBE language (and system) originated from TU Delft research. Reinier van Dijk* is founder of ParaPy BV

A genealogy of KBE systems



The KBE programming approach

To define complex product models with modern KBE systems, it is not necessary to be an expert software developer.

KBE languages are designed to be used primarily by engineers and domain experts, not only by software specialists!

Hide software complexity, expose engineering intent!

Disclaimer: KBE languages are **engineer-oriented,** but still require **structured programming thinking**

KBE constructs to define class and object hierarchies

KBE languages allow engineers to formalize domain knowledge through dedicated **product model definition constructs** that:

- Define a class and its superclasses
- Define attributes
- Define behaviour (e.g. methods)
- Define component structure (aggregation/composition)*
- Establish dependencies with other classes and their attributes

These constructs allow engineers to assemble product models of arbitrary complexity while mapping onto the underlying OO mechanisms and hiding much of the software complexity behind the scenes (e.g. dependency tracking, lazy evaluation, memory management).

KBE constructs to define class and object hierarchies

Examples of equivalent product model definition constructs from various KBE languages:

KBE Language	Construct
GDL	`define-object`
IDL	`defpart`
Knowledge Fusion	`DefClass`
AML	`define-class`
ParaPy	`class … (Base)`

The Para*Py* product model definition construct

`class…(Base)`

The ParaPy product model definition construct

Let's recall the concept of **class-frame**.
Imagine we want to model this:

Aircraft	
attribute 1	float
attribute 2	some_value
attribute 3	[method]
…	
has-part	→ Wing, Fuselage

Wing	
span	float
sweep	35 deg
Wing_attr	[method]
…	
has-part	→ WingSurface, WingStructure

Fuselage	
diameter	
length	

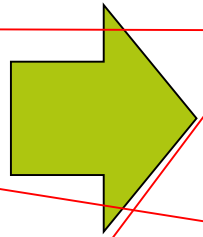
The ParaPy product model definition construct

Aircraft	
attribute 1	float
attribute 2	some_value
...	
attribute 3	[method]
has-part	→ Wing, Fuselage

- Required input slots
- Optional input slots

- Computed slots

Definition of the assembly components (**parts**) via links to other classes



```
class Aircraft(Base):
    #: required input slot
    attribute1 = Input()
    #: optional input slot
    attribute2 = Input(some_value)
    ...
    @Attribute
    def attribute3(self):
      return ...

    @Part
    def my_wing(self):
      return Wing(span=10)

    @Part
    def my_fuselage(self):
      return Fuselage(diameter=2, length=8)
```

ParaPy

The ParaPy product model definition construct

```
class Aircraft(Base):
    #: required input slot
    attribute1 = Input()
    #: optional input slot
    attribute2 = Input(some_value)
    ...
    @Attribute
    def attribute3(self):
        return ...

    @Part
    def my_wing(self):
        return Wing(span=10)

    @Part
    def my_fuselage(self):
        return Fuselage(diameter=2, length=8)
```

ParaPy

By instantiating the class **Aircraft**, an object will be created (e.g. **my_aircraft**).
This object will be an assembly composed of two other objects (*parts*):

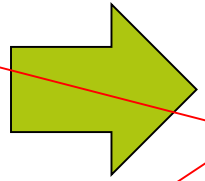
- **my_wing**
- **my_fuselage**

The object **my_wing** will be an instance of the class **Wing**

The object **my_fuselage** will be an instance of the class **Fuselage**

The ParaPy product model definition construct

Wing	
span	float
sweep	35 deg
wing_attr	[method]
…	
has-part	→ WingSurface, WingStructure



```
class Wing(Base):
 #: required input slot
span=Input()
 #: optional input slot
sweep=Input(35)
@Attribute
def wing_attr(self):
  return
```

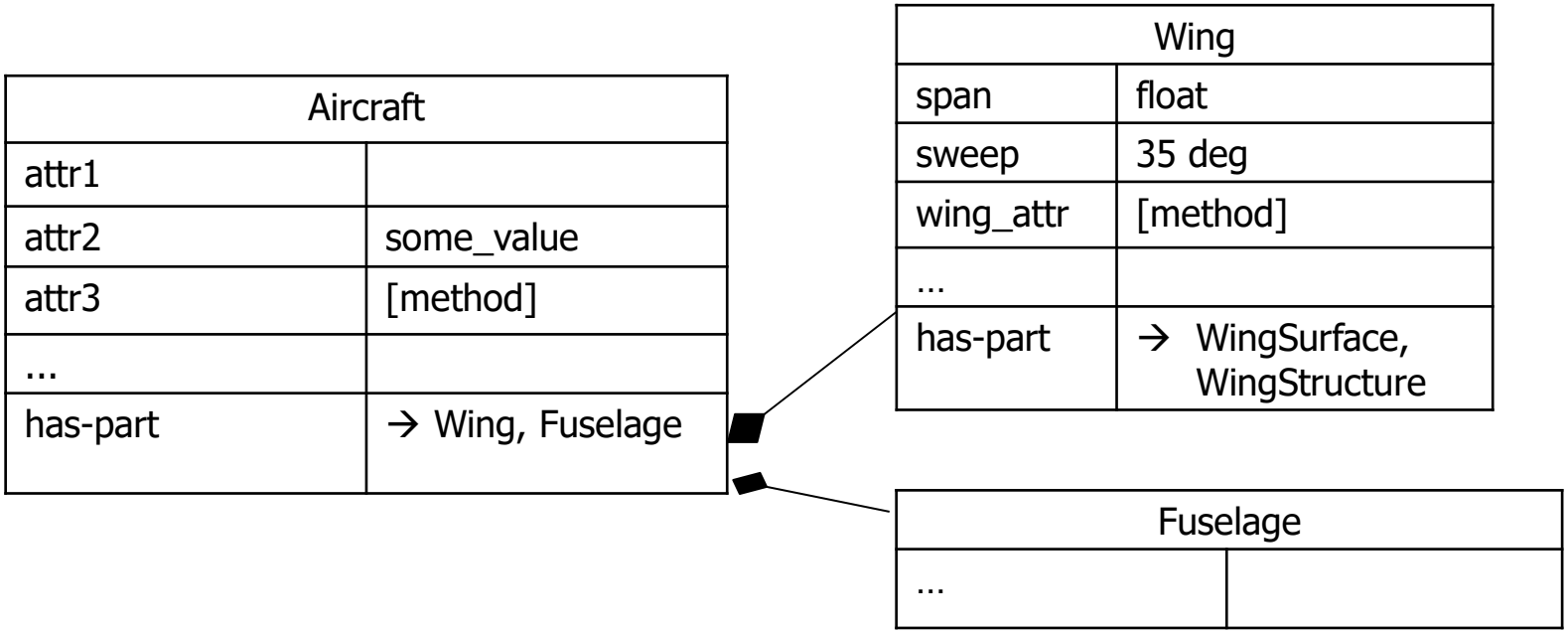
```
@Part
def my_wing_surface(self):
  return WingSurface(span=self.span)

@Part
def my_wing_structure(self):
  return WingStructure(material="CFRP")
```

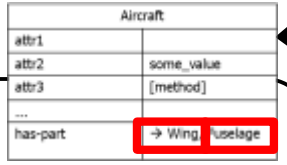
Definition of slots
- Required input slots
- Optional input slots
- Computed slots

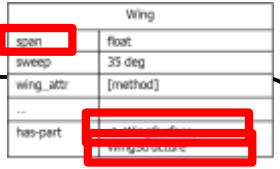
Definition of the assembly components (**parts**) via links to other classes

The ParaPy product model definition construct



The ParaPy product model definition construct






```
class Aircraft(Base):
    #: required input slot
    attr1 = Input()
    #: optional input slot
    attr2 = Input(some_value)
    ...
    @Attribute
    def attr3(self):
      return ...
      @Part
    def my_wing(self):
      return Wing(span=10)
    @Part
    def my_fuselage(self):
      return Fuselage(diameter=2, length=8)
```

```
class Wing(Base):
    #: required input slot
    span=Input()
    #: optional input slot
    sweep=Input(35)
    @Attribute
    def wing_attr(self):
      return
    @Part
    def my_wing_surface(self):
      return WingSurface(span=self.span)
    @Part
    def my_wing_structure(self):
      return WingStructure(material="CFRP")
```

The ParaPy product model definition construct

DISCLAIMER:

The classic KBE product model definition constructs, e.g. the ParaPy Class ..(Base), recall the frame concept for their structural similarities.

However, there is a fundamental difference between the two execution paradigms:

- Frame based system → Inference (with method to support slot evaluations)
- KBE → Dependency computations with no inference*

KBE systems are NOT frame-based systems

UML modeling exercise

Make a **UML class diagram** to model the entities and relationships captured in the KBE app snippet below:

```
class Aircraft(Base):
    #: required input slot
    attr1 = Input()
    #: optional input slot
    attr2 = Input(value)
    ...
    @Attribute
    def attr3(self):
        return ...
    @Part
    def my_wing(self):
        return Wing(span=10)
    @Part
    def my_fuselage(self):
        return Fuselage(diameter=2, length=8)
```

```
class Wing(Base):
    #: required input slot
    span=Input()
    #: optional input slot
    sweep=Input(35)
    @Attribute
    def wing_attr(self):
        return
    @Part
    def my_wing_surface(self):
        return WingSurface(span=self.span)
    @Part
    def my_wing_structure(self):
        return WingStructure(material="CFRP")
```

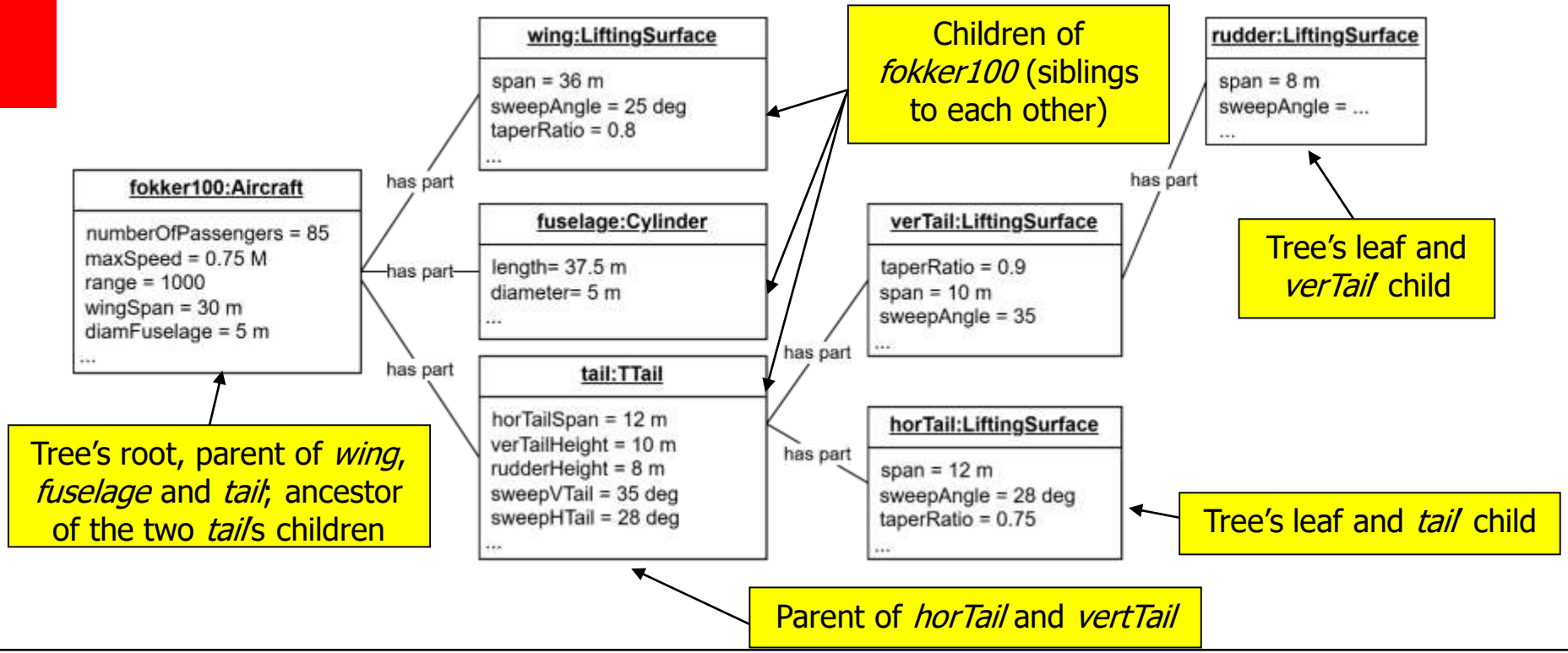
KBE app

UML Object diag.

UML Class diag.

UML modeling exercise

This is a UML **object diagram**

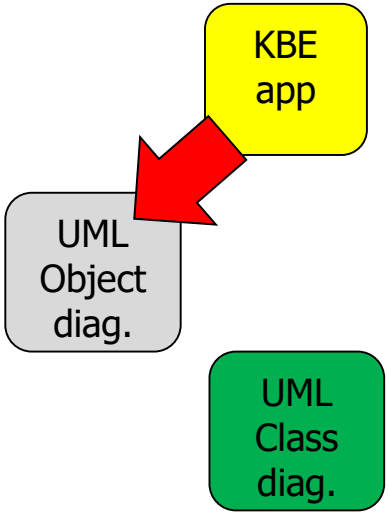


UML modeling exercise

Make a **UML object diagram** to represent a possible instantiation of the KBE app snippet below:

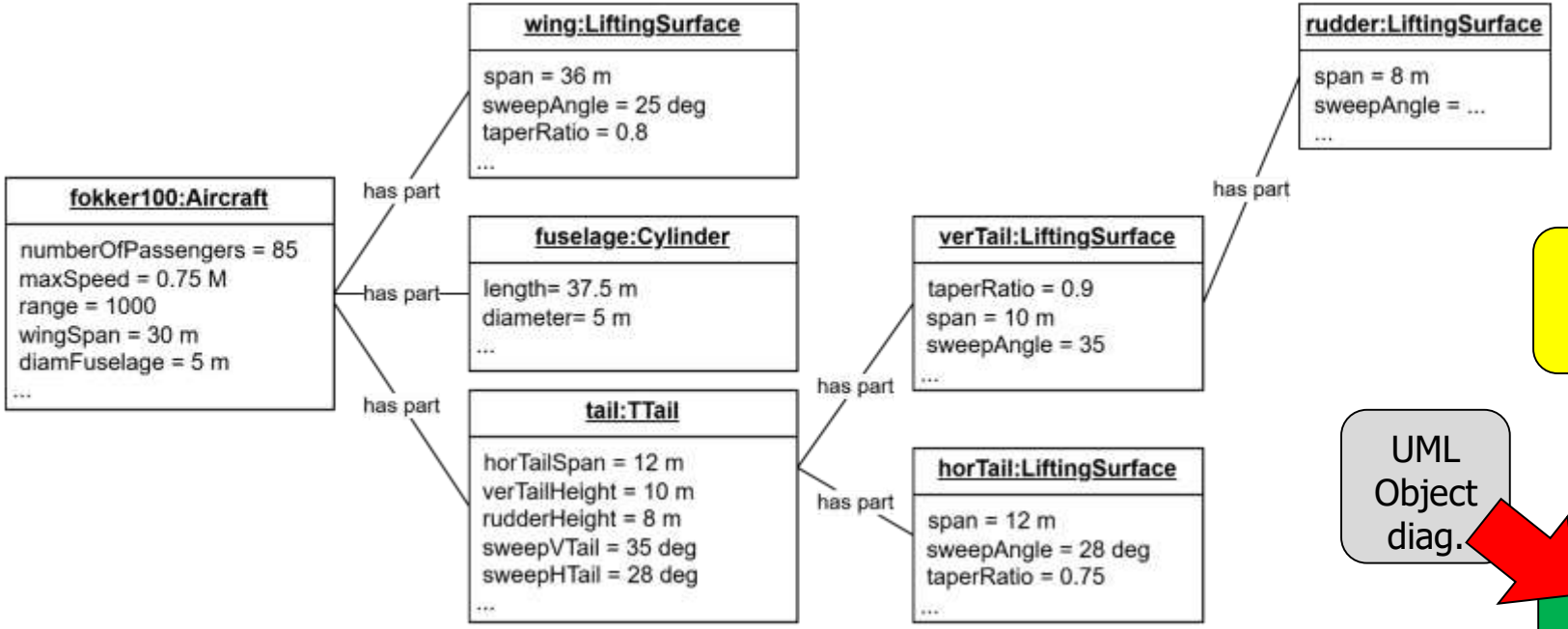
```
class Aircraft(Base):
    #: required input slot
  attr1 = Input()
  #: optional input slot
  attr2 = Input(value)
  ...
  @Attribute
  def attr3(self):
    return ...
    @Part
  def my_wing(self):
    return Wing(span=10)
  @Part
  def my_fuselage(self):
    return Fuselage(diameter=2, length=8)
```

```
class Wing(Base):
    #: required input slot
  span=Input()
    #: optional input slot
  sweep=Input(35)
  @Attribute
  def wing_attr(self):
    return
  @Part
  def my_wing_surface(self):
    return WingSurface(span=self.span)
  @Part
  def my_wing_structure(self):
    return WingStructure(material="CFRP")
```



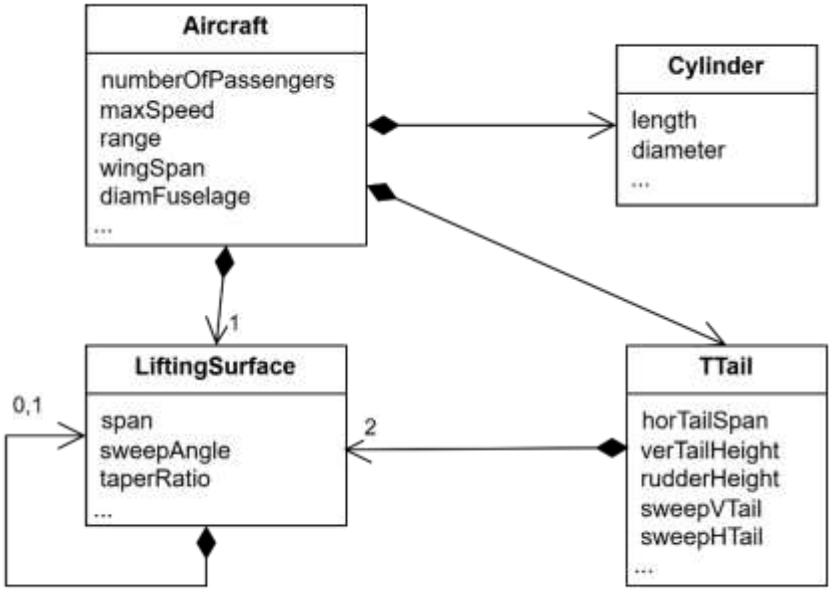
UML modeling exercise

Build the UML **class diagram** corresponding to the object diagram below:



UML modeling exercise

UML **class diagram** corresponding to the object diagram from previous slide:



KBE app

UML Object diag.

UML Class diag.

The Para*Py* class definition

The Para*Py* *class* definition. Basic structure

```
class ClassName(Base, OtherSuperclass...):
    #: required input slot
    required_input_1=Input()
    #: optional input slot
    input_with_default_value_1=Input(value)

    @Attribute
    def attribute_slot_1(self):
        return

    @Part
    def component_name_1(self):
        return OtherClassName(input_value=value)
```

The name assigned to the **class** being defined here

The collection of classes (**superclasses**) from which the class here specified **inherits** attributes and components. It can be empty*, e.g. ClassName()

Sequence of **required** input slots, i.e. necessary to create instances of the given class

Sequence of other input slots, for which a **default value** can be provided

All the above form the ***input*** of the class

...while these form the ***output*** of the class

The ParaPy *class* definition. Basic structure

```
class ClassName (Base, OtherSuperclass…):

  #: required input slot
  required_input_1=Input()

  #: optional input slot
  input_with_default_value_1=Input(value)

  @Attribute
  def attribute_slot_1(self):
    return

  @Part
  def component_name_1(self):
    return OtherClassName(input_value=value)
```

The ***@Attribute* slots** (or computed slots) are defined as expressions which are evaluated at runtime

These expressions can either be IF-THEN rules or any other mathematical, logic or engineering rule (many examples later)

The arguments of these expressions can be values of

- any type of input slots
- any other attribute
- any slot of any of the class' parts (*@Part*)
- Any slot of any descendant or ancestor object in the instantiated object tree

The ParaPy *class* definition. Basic structure

```
class ClassName (Base, OtherSuperclass...):

  #: required input slot
  required_input_1=Input()

  #: optional input slot
  input_with_default_value_1=Input(value)

  @Attribute
  def attribute_slot_1(self):
    return

  @Part
  def component_name_1(self):
    return OtherClassName(input_name_1=value_1,
                          input_name_n=value_n)
```

The ***@Part* slots** define the **sequence of objects** (parts) that will be components of any instance of **ClassName***

For each object (*@Part*), the following must be specified:

- Name (**component_name_1**) of the object
- Name of the class (**OtherClassName**) to instantiate for generating the given object (by using the keyword **return**)
- List of input slot names (**input_name_...**) & corresponding values (**value_...****).
- The names of the input must match those of the input slots of the class to be instantiated.
- The length of this list must match at least the amount of **required input slots** of the class to be instantiated.

Main reference material for this lecture

1. **Book** *MDO supported by KBE* by Wiley**: Chapter 9 on KBE**
2. **Journal paper** *Knowledge based engineering: Between AI and CAD. Review of a language based technology to support engineering design,*
   G. La Rocca,, Journal of Advanced Engineering Informatics, Vol. 26, pp.159–179, 2012.
3. **On UML** (*Object Oriented modeling Class and Object diagrams; A summary of UML notation)*
4. **Para*Py* documentation***

Ref 2-3 available in the section Reading Material